

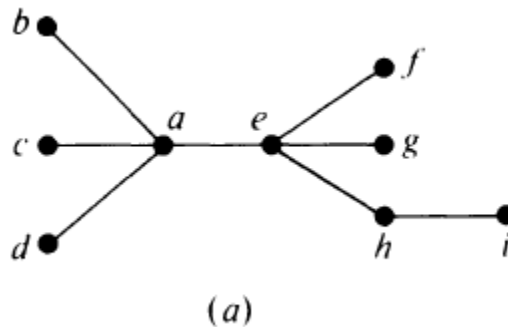
# Trees and Cut Set

# Outline

- Introduction
- Properties of a tree
- Characteristics of a tree
- Rooted trees
- Path lengths in rooted trees
- Prefix codes
- Spanning trees and Cut-sets
- Transport networks

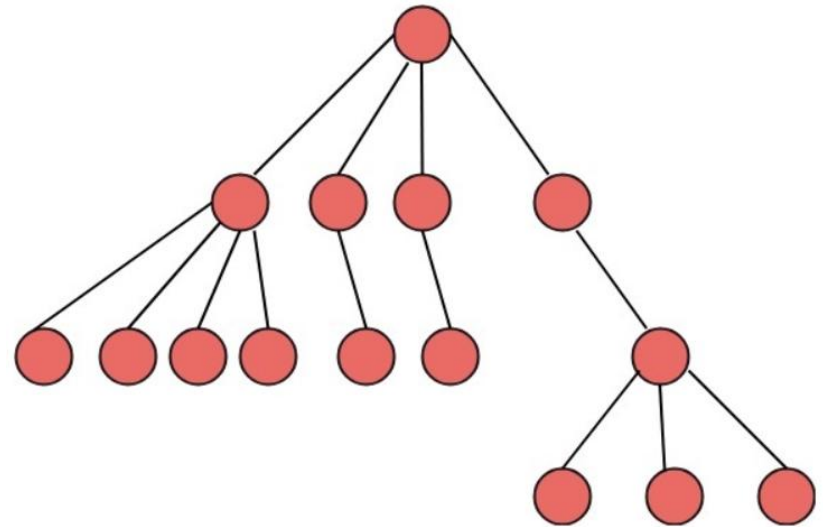
# Introduction

- **Tree** is a discrete structure that represents hierarchical relationships between individual elements or nodes.
- A Tree is a connected acyclic undirected graph.



# Introduction

- Terminal / Leaf node
  - Degree of vertex is 1
- Branch / Internal node
  - Degree of vertex is more than 1.



# Property -1

- **There is a unique path between every two vertices in the tree.**
- Above property hold because tree is connected and it has no cycles.

# Property -2

- The number of vertices is one more than the number of edges in a tree.
- $e = v - 1$
- Proof by Induction : Self study.

# Proof by Induction : Self study.

- **Theorem 3:** Prove that a tree with  $n$  vertices has  $(n-1)$  edges.

**Proof:** Let  $n$  be the number of vertices in a tree  $(T)$ .

If  $n=1$ , then the number of edges=0.

If  $n=2$  then the number of edges=1.

If  $n=3$  then the number of edges=2.

Hence, the statement (or result) is true for  $n=1, 2, 3$ .

Let the statement be true for  $n=m$ . Now we want to prove that it is true for  $n=m+1$ .

Let  $e$  be the edge connecting vertices say  $V_i$  and  $V_j$ . Since  $G$  is a tree, then there exists only one path between vertices  $V_i$  and  $V_j$ . Hence if we delete edge  $e$  it will be disconnected graph into two components  $G_1$  and  $G_2$  say. These components have less than  $m+1$  vertices and there is no circuit and hence each component  $G_1$  and  $G_2$  have  $m_1$  and  $m_2$  vertices.

$$\begin{aligned}\text{Now, the total no. of edges} &= (m_1-1) + (m_2-1) + 1 \\ &= (m_1+m_2)-1 \\ &= m+1-1 \\ &= m.\end{aligned}$$

- Hence for  $n=m+1$  vertices there are  $m$  edges in a tree  $(T)$ . By the mathematical induction the graph exactly has  $n-1$  edges.

# Property -3

- A tree with two or more vertices has at least two leaves.
- Proof on next slide.



~ A tree with  $n$  vertices ( $n \geq 2$ ), contains at least 2 leaves.

Total vertices =  $n$

leaves =  $k$

non-leaves =  $n - k$

Sum of  
Degree of  
vertices

$$\sum_i d(v_i) = k(1) + (n-k) \cdot \text{degree}$$

$$\geq k + 2(n-k)$$

$$\geq 2n - k$$

non leaf vertex  
has at least degree  
2

$$\begin{aligned} \text{also, } \sum_i d(v_i) &= 2 \times e \\ &= 2 \times (n-1) \quad [e = n-1] \\ &= 2n - 2 \end{aligned}$$

$$\text{so, } 2n - 2 \geq 2n - k$$

$$\boxed{k \geq 2}$$

Characteristics of a tree

# Characteristics-1

- **A graph in which there is a unique path between every pair of vertices is a tree.**

To show characterization 1, we note first that a graph in which there is a path between every pair of vertices is connected. Moreover, the graph cannot contain a circuit if these paths are unique, since the existence of a circuit implies the existence of two distinct paths between a certain pair of vertices. Thus, we can conclude that a graph in which there is a unique path between every pair of vertices is a tree.

# Characteristics-2

- **A connected graph with  $e = v - 1$  is a tree.**
- Suppose a connected graph on  $v$  vertices has  $v-1$  edges, we must prove no cycle exists, suppose it does, if we remove an edge from the cycle we get a connected graph (because if we have a path that uses this edge, instead of using the edge we can go around the remaining part of the cycle). This is a contradiction, since the graph would have  $v-2$  edges, and would hence not be connected.

# Characteristics-3

- **A graph with  $e = v - 1$  that has no circuit is a tree.**
- **Proof : Next slide**

~ Let's assume Graph is not Connected

It is disconnected and has  $k$  Components

1<sup>st</sup> Component  $e_1 = v_1 - 1$

2<sup>nd</sup> Component  $e_2 = v_2 - 1$

$\vdots$   
 $e_k = v_k - 1$

$$e_1 + e_2 + \dots + e_k = (v_1 + v_2 + \dots + v_k) - (1 + 1 + \dots + k)$$

$$\text{So, } e = v - k$$

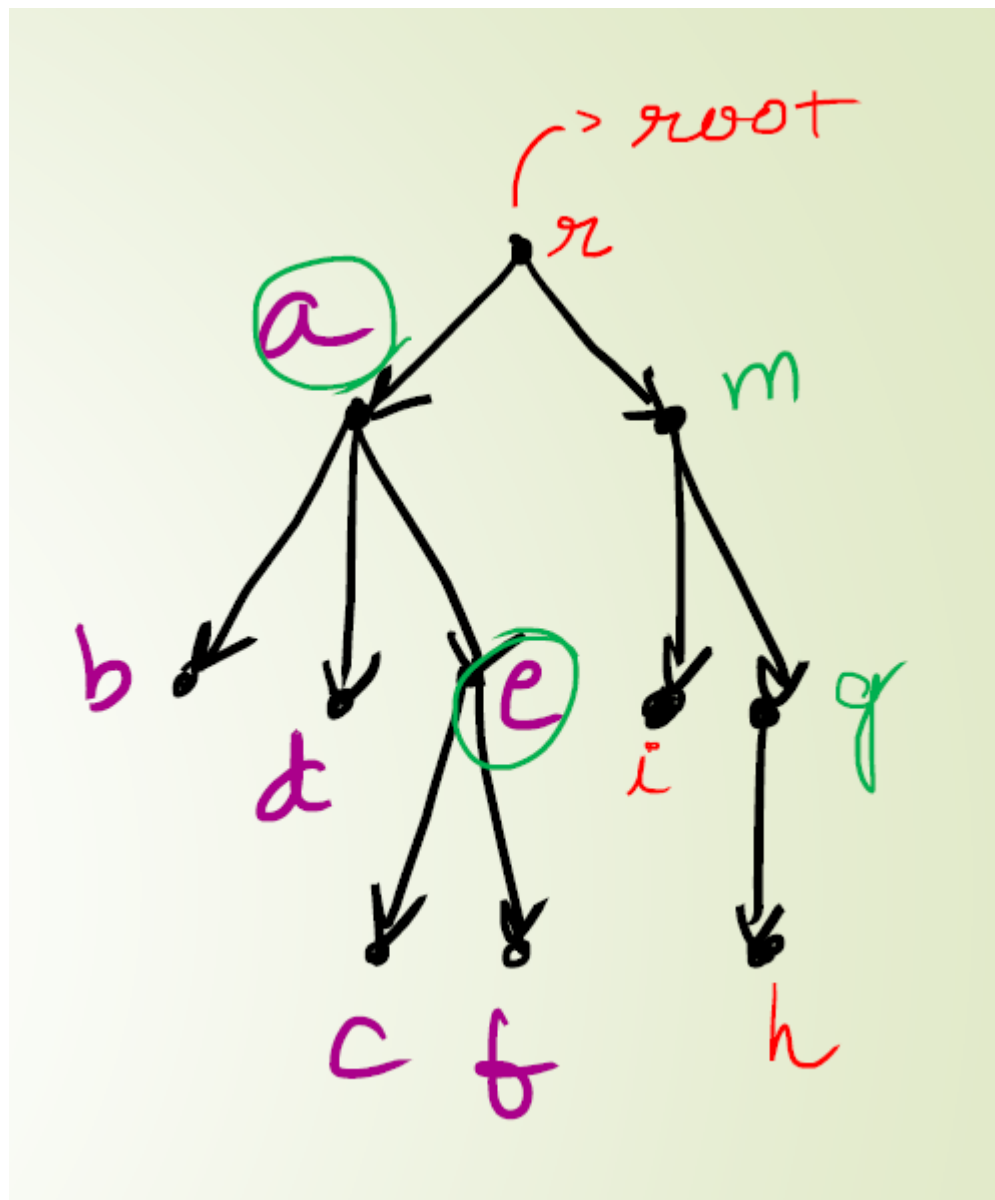
But  $v - k < v - 1$

So,  $e < v - 1$  which is

Contradiction. So, Graph is Connected.

# Rooted Trees

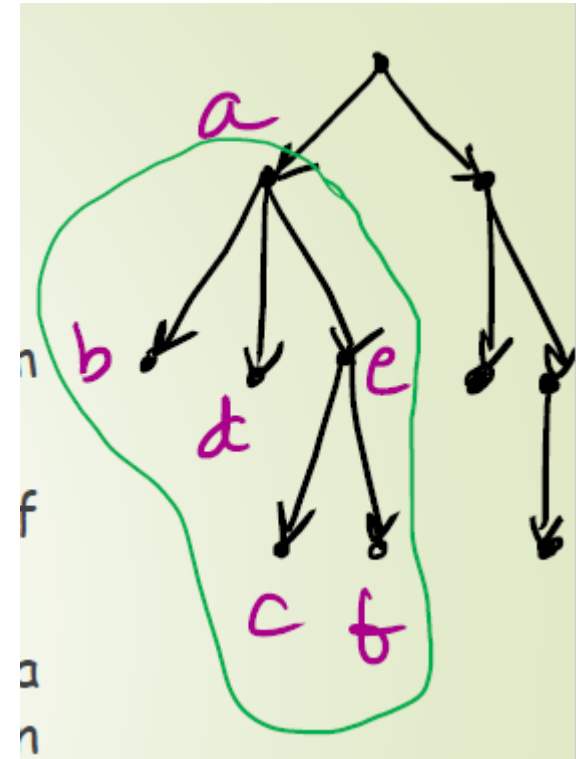
- A digraph is said to be directed tree if it becomes a tree when the directions of the edges are ignored.
- A directed tree is called a rooted tree if there is exactly one vertex whose incoming degree is 0 and outgoing degree is 1.
- In a directed tree, a vertex whose outgoing degree is 0 is called a leaf or terminal node, and a vertex whose outgoing degree is non zero is called a branch node or an internal node.





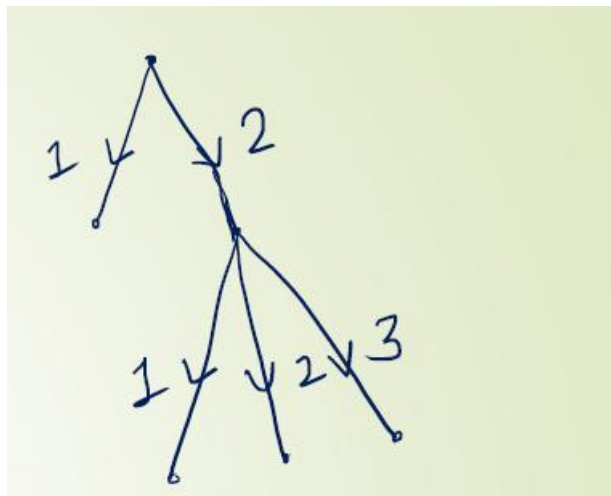
# Rooted Trees

- $a$  is branch node.
- A vertex  $b$  is said to be children of  $a$  if there is an edge from  $a$  to  $b$  also,  $a$  is said to be father of  $b$
- Two vertices are said to be siblings if they are children of the same vertex.
- Vertex  $c$  is said to be a descendant of  $a$  if there is a directed path from  $a$  to  $c$  also,  $a$  is said to be an ancestor of  $c$ .
- Let  $a$  be a branch node in the tree  $T$ . by the subtree with  $a$  as the root means the subgraph  $T' = (V', E')$  of  $T$  such that  $V'$  contains  $a$  and all of its descendants and  $E'$  contains the edges in all directed paths emanating from  $a$



# Rooted Trees

- An ordered tree is rooted tree with the edges incident from each branch node labeled with integers  $1, 2, \dots, i$ .
- Two ordered trees are said to be isomorphic if there is one to one correspondence between their vertices and edges that preserves the incident relation and if the labels of corresponding edges match.



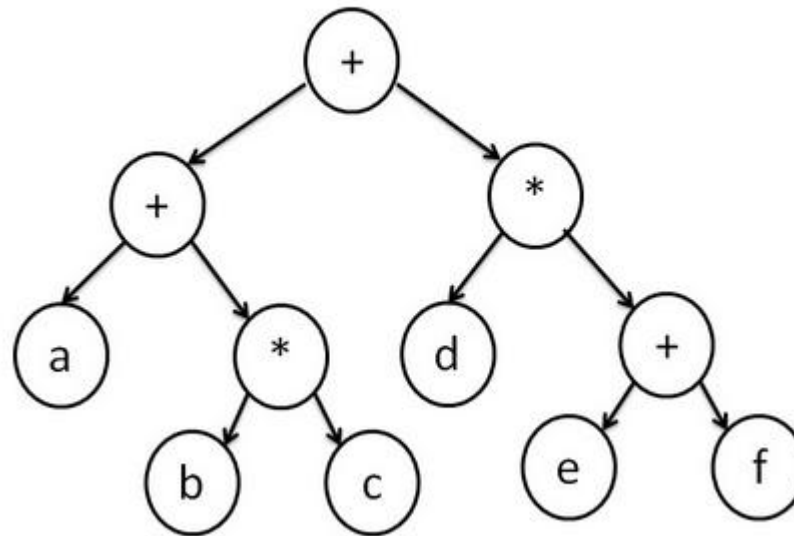
# Rooted Trees

- An ordered tree in which every branch node has at most  $m$  children is called an  $m$  ary tree.
- An  $m$  ary tree is said to be regular if everyone of its branch nodes has exactly  $m$  children.

# Expression Tree

- Expression Tree is used to represent expressions.

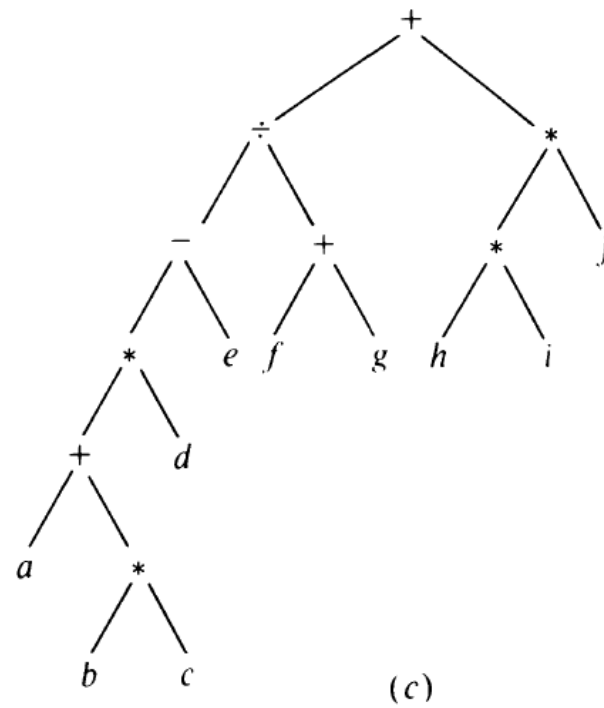
$a + (b * c) + d * (e + f)$



# Assignment

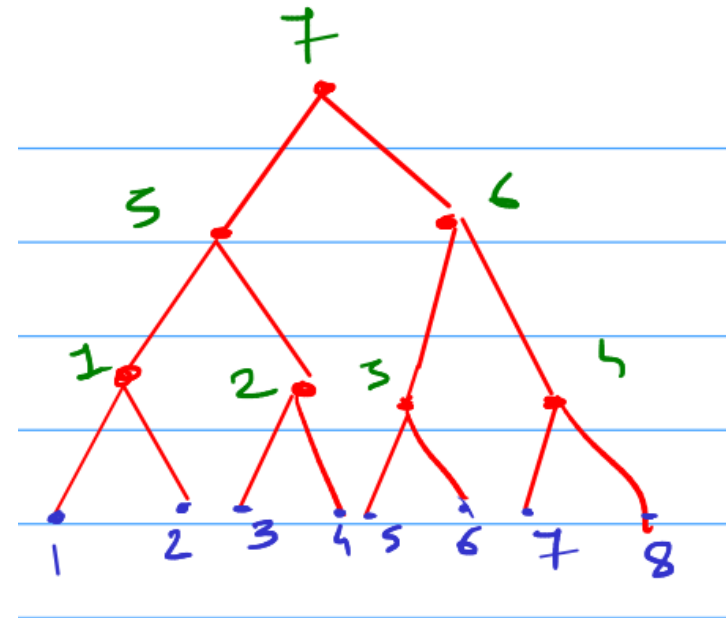
$$(((a+b*c)*d-e) / (f+g)) + h*i*j$$

# Solution



# Path lengths in rooted tree

- $i$  = number of branch/internal nodes
- $t$  = number of terminal nodes of regular binary trees
  - $i = t - 1$
- Extending above case for  $m$ -ary trees
  - $(m-1)i = t - 1$



**Example 6.1** Consider the problem of connecting 19 lamps to a single electric outlet by using extension cords each of which has four outlets. Since any such connection is a quaternary tree with the single outlet connected to the root of the tree, according to (6.2),



**Example 6.1** Consider the problem of connecting 19 lamps to a single electric outlet by using extension cords each of which has four outlets. Since any such connection is a quaternary tree with the single outlet connected to the root of the tree, according to (6.2),

$$(m-1)i = t-1$$

$$m = 4$$

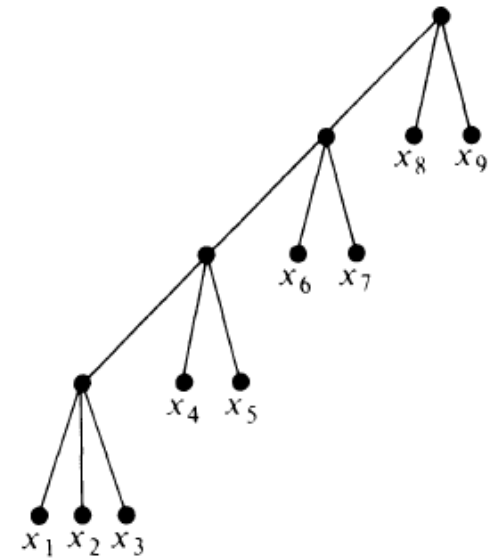
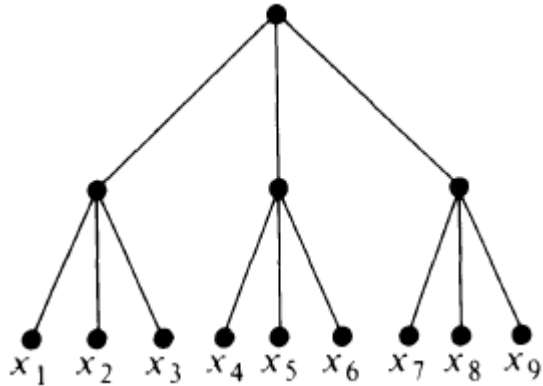
$$t = 19$$

$$\text{So, } i = 18/3 = 6$$

Hence, we need 6 cords.

# Path lengths in rooted tree

- The path length of a vertex in a rooted tree is defined to be the number of edges in the path from the root to the vertex.
- Height of the tree is maximum of the path lengths in the tree.



m-ary tree of height h has at most  $m^h$  leaves in the type of tree as shown in figure 1. (  $3^2 = 9$  )

m-ary tree of height h has at least  $m+(m-1)(h-1)$  leaves for the type of tree shown in figure 2. (  $3 + (3-1)*(4-1)$  )

# Path lengths in rooted tree

- $I$  = sum of the path lengths of all branch nodes
- $E$  = sum of the path lengths of leaves

$$E = I + 2i$$

(  $i$  is number of internal nodes)

- For  $m$ -ary tree,

$$E = (m-1)I + mi$$

# Prefix Code : Application of tree

- A prefix code is the binary code structure that represents certain letters or characters. These code strings are used by computers to communicate. They consist of zeros and ones that translate into certain commands or words.

$\{00, 01, 0001\}$   $\leftarrow$  Not Prefix Code

$\{000, 10, 11, 001, 01\}$   
 $\nwarrow$  Prefix Code

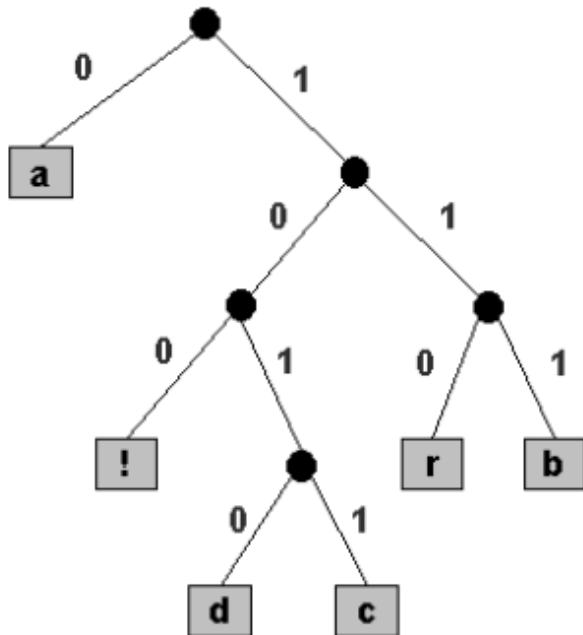
# Prefix Code

- Binary trees are used to represent how strings of numbers in prefix code translate to certain letters, characters or messages. Many software applications use a prefix code based on binary trees to compress their data.
- Example : Huffman encoding technique always generates prefix codes.

# Prefix Code

- A prefix code is most easily represented by a binary tree in which the external nodes are labeled with single characters that are combined to form the message. The encoding for a character is determined by following the path down from the root of the tree to the external node that holds that character: a 0 bit identifies a left branch in the path, and a 1 bit identifies a right branch. In the following tree, black circles are internal nodes and gray squares are external nodes. The code for b is 111, because the external node holding b is reached from the root by taking 3 consecutive right branches. The other codes are given in the table below.

# Prefix Code



| character | encoding |
|-----------|----------|
| a         | 0        |
| b         | 111      |
| c         | 1011     |
| d         | 1010     |
| r         | 110      |
| !         | 100      |



# Spanning Trees

- A tree of a graph is a subgraph of the graph which is a tree.
- A spanning tree of a connected graph is a spanning subgraph of the graph which is a tree.
- A branch of a tree is an edge of the graph that is in the tree.
- A chord, or a link of a tree is an edge that is not in the tree.
- The set of chords of a tree is referred to as the complement of the tree.

